

Deep Monkey Studio：面向工业数字孪生构建、运行与交付的一体化平台

张文鹏 (Wenpeng Zhang)

独立研究者

15184552744@163.com

中文工作稿 • 2026 年 10 月 9 日

代码: <https://github.com/fatasia/DeepMonkey-Studio>

研究状态: 本文描述已有实现、渲染测量记录及针对性软件测试。贯穿资产全生命周期的对照实验尚待完成。

摘要

工业数字孪生需要连接不同来源的工程模型、实时设备数据、交互图形与业务逻辑。当模型发生修订、用户与智能体同时编辑，或运行过程被中断时，跨层维护资产身份与状态会变得困难。Deep Monkey Studio 将模型导入、工业连接、二维与三维构建、WebGPU 渲染引擎、本体约束动作、物理仿真和原生交付集成在同一平台中。其架构连接了带版本的资产、经过校验的场景事务、持久化智能体检查点，以及可执行的发布产物。本文介绍已实现的平台，并通过 86 项针对性软件测试检查其中的部分机制。已有渲染对照也呈现出明确的性能取舍：在包含 1,000 个对象的测试场景中，Deep WebGPU 后端报告了 7 次绘制调用，但首帧延迟和动态场景帧时间仍有优化空间。本文设计了四类生命周期实验，用于测量身份保持、编辑正确性、恢复能力与跨运行时一致性。核心问题是：通过显式身份和版本检查组织平台集成，能否减少工业 workflows 中的错误与集成成本。

1 引言

工业数字孪生应在首次演示之后继续发挥作用。CAD 装配更新后，设备绑定应得到保留；传入的测量值应更新正确的资产；维护动作应保留无关的编辑；交付应用应呈现已经批准的场景。这些要求涉及图形、数据集成、语义、执行和部署多个层面。

Deep Monkey Studio 用一个平台承载这套工作流。构建环境集成了场景编辑、仪表板、脚本、插件和工业智能体。共享接口契约将这些工具与模型转换、设备数据、仿真以及浏览器和原生运行时连接起来。工程目标是在保持扩展能力和本地部署能力的同时，让资产整个生命周期中的变更可以追溯。

系统设计分为三个部分：以资源和着色器契约组织的模块化渲染与执行引擎；连接工程资产、设备数据、编辑器和领域动作的可编程构建平台；连接构建与浏览器、原生交付的版本化场景产物。本文说明其接口与设计选择，再分析已有软件行为和渲染证据。电池预测算法由配套论文讨论，在本文中作为使用平台接口契约的领域运行时示例。

研究问题。 RQ1: 导入、修订、编辑和交付过程中，资产身份与用户覆盖设置能在多大程度上保留？RQ2: 在人与智能体并发操作时，版本检查与语义动作准入能否减少错误编辑？RQ3: 这些机制带来了怎样的恢复、渲染与集成成本？

2 相关工作

Holodeck 通过资产与空间约束研究语言引导的具身环境生成 [1]。SceneCraft 将结构化场景生成与程序化工具结合 [2]；RoboGen 将生成的任务和环境连接到机器人技能学习 [3]。Code2Games 将场景和玩法表示与引擎反馈、行为反馈联系起来 [4]。这些工作为本文采用显式表示和可执行评价提供了参考。

已有数字孪生软件也不局限于可视化演示。Eclipse Ditto 提供设备状态中间件 [5]；iTwin.js 提供基础设施数字孪生开发库 [6]；OpenTwins 结合了三维可视化、物联网数据流和机器学习预测 [7]。因此，本文关注 workflow 覆盖、依赖要求、状态保持和实测运行成本。仅仅完成一体化实现，还不足以说明平台优于这些系统。

3 平台架构

层次	职责与接口
输入	工程文件、设备协议、数据库查询和事件流；源身份与格式能力配置。
资产与数据	带版本的模型包、层级与属性、规范化数据行和事件、对象绑定。
构建	二维与三维编辑器、仪表板、脚本、插件、场景事务，以及人与智能体使用的工具。
运行	WebGPU 渲染、本体动作、持久化智能体任务、无界面物理仿真及领域服务。
交付	编译后的场景产物、运行时包、浏览器与原生消费方、内容哈希及运行证据。

实现结合了 TypeScript/React 应用代码与 Rust/原生组件。现有 Three.js 查看器与 Deep WebGPU 后端并存。与引擎无关的身份标识和版本化资源包连接构建与执行；各适配器都有明确的支持范围。

3.1 Deep Engine：渲染、资源与跨端执行

Deep Engine 提供资源包、实例化、渲染调度、着色器包和 WebGPU 执行。渲染实现包括 GGX 材质、基于图像的光照、方向光阴影、线性 HDR、色调映射、透明处理和后处理。glTF 路径处理受支持的纹理通道、节点动画、骨骼蒙皮和形态目标；遇到未支持的编码特性时返回结构化错误。

资源驻留管理对几何和纹理使用统一预算。可见块与预取块按需请求细节层级；候选资源先经过校验，再在帧边界发布。取消请求和被新请求替代的请求分别控制待执行上传与清理。着色器包记录入口点、绑定、顶点布局和渲染通道配置，并使用 TypeScript/Rust 共享表示。版本化二维显示列表将路径、文字、图片、裁剪与命中区域传递给原生图表渲染。

这些机制支撑本地交付与后端演进。整个产品的视觉等价性、设备丢失恢复，以及完整动画和物理能力覆盖，需要独立于隔离渲染基准开展验收。

调度与资源复用。 渲染图检查资源依赖并按拓扑次序执行，同时生成临时资源的生命周期和物理槽计划。跨资源复用要求格式、尺寸、采样数与使用方式相容，外部资源不进入临时槽。这使资源复用成为显式调度决策。大场景驻留规划保留可见对象可绘制的最粗细节层，再按预算追加几何细节和纹理 mip；被新请求替代的上传通过所有权和清理规则退出。

可编程材质与执行接口。 DeepSL surface 编译路径将 Standard/Unlit 材质属性转换为类型化中间表示与 WGSL。版本化着色器包描述入口、绑定、顶点流、附件及渲染状态，由浏览器与 Rust 执行器校验并创建管线。缓存区分可持久化内容与设备相关对象。场景状态与后端分离，宿主控制绘制节奏和后端切换，编辑器、独立 SDK 与原生播放路径由此使用相应接口。

3.2 完整创作工作流与扩展

平台将二维页面、三维场景、数据绑定、脚本和插件组织在同一项目中。模型层级和属性用于查找与选择对象，设备数据为绑定对象提供状态，仪表板呈现相同项目的数据，领域服务为动作和分析提供计算。引擎承担视觉与运行时执行，编辑器提供交互入口；场景事务和发布产物连接编辑与部署。这样的分工让渲染引擎可以作为模块使用，同时保留完整的创作应用。

功能完整性应按任务路径呈现：接入工程模型和设备数据、建立对象绑定、编辑场景与页面、执行领域动作、保存并交付。模型格式和协议的广度决定输入覆盖；脚本、插件和 SDK 决定扩展方式；运行时包和本地部署决定交付方式。下文分别列出已查阅的输入路径、操作与测量范围。

3.3 模型导入与保真

导入过程保留源文件信息，并生成附带层级和属性的运行时几何。格式覆盖以版本、支持子集和保留信息描述。表 1 总结了已检查的实现路径与文档，并非完整的兼容性认证。

表 1: 工程模型路径与支持范围。

格式	已实现路径或已验证范围
glTF / GLB	运行时交换、优化与发布；扩展支持范围由具体加载器决定。
IFC	通过 fragments 导入 BIM，保留源表示与语义访问能力。
STEP / STP	通过开源曲面离散化生成运行时几何，并附带层级与属性文件。
FBX; DXF	分别通过对应适配器完成场景与动画加载、二维图纸导入。
OpenUSD	支持已检查样本中的 USDA/USDC/USDZ 加载；复杂组合仍需单独验证。
JT	内置路径覆盖已检查的 9.5/10.3 配置中的 LOD0 三角几何、装配实例及部分属性与材质；PMI 结构解析与语义标注解析分开评价。
X_T	对限定几何子集提供内置文本解析。修剪、曲面类型、命名、颜色和装配信息的损失取决于具体配置。

更广泛的格式目录还记录了候选格式。目录中出现某格式，不代表导入器已经可用。原生专有格式须有通过准入的内置或开源实现路径，才能列入实际运行覆盖矩阵。评价应分别检查几何、装配结构、属性、交互和交付。

3.4 工业协议与数据源

API 调用实际的连接器适配器，并将返回数据规范化后提供给消费方。持续订阅与限定范围的预览读取满足不同的运行需求。

接口类别	已检查的操作
MQTT; OPC UA	持续订阅与预览读取; 共享订阅生命周期管理; 处理 OPC UA 质量、时间戳和配置的安全选项。
Modbus TCP; BACnet; S7; EtherNet/IP	在限定范围内读取寄存器、线圈、属性、PLC 地址或标签。
HTTP; WebSocket; Kafka; AMQP	通过各自适配器发起请求或进行限定范围的消息采样。
SNMP; CoAP; TCP; UDP; 串口	针对不同数据源的采样适配器。
SQL 与分析存储	PostgreSQL、MySQL 兼容存储、SQL Server、Oracle、ClickHouse 和时序数据源等查询适配器。
文档与文件数据源	MongoDB、Elasticsearch、CSV 和 Excel 适配器。

持续运行的 MQTT 与 OPC UA 数据源向事件基础设施提供数据。仿真结果可通过 MQTT 和 SQL 路径导出。实际设备覆盖取决于传输配置、地址映射、数据类型与厂商行为; 现场评价应同时记录这些条件、重连结果和事件顺序。

3.5 资产修订与受控编辑

源哈希与资源包身份将修订关联到具体资产。重导入协调器在发布前准备更新, 支持回滚、代次检查和冲突处理。产品适配器更新场景快照, 同时保留身份关系及适用的覆盖设置。

场景事务携带基准版本和经过校验的命令。事务层拒绝基于过期版本的写入, 并在部分执行、错误或取消后调用回滚。编辑器与智能体消费方共用该路径; API 与浏览器之间的桥接层保留事务身份并缓存结果。这为并发操作提供了明确的准入条件。

3.6 本体、智能体与仿真

本体动作将规范目标绑定到已发布的定义、参数、作用范围和前置条件, 执行后生成回执。属性约束分为仅报告和严格执行两种行为, 因此每次实验都需要记录所选模式。

已接入产品的工业智能体运行时使用工具网关、持久化检查点、取消机制和基于版本的恢复。Web 工作区提供恢复与取消操作。另一个独立的任务图辅助模块支持次数受限的修正, 但本次核查尚未找到它的产品消费方。

World API 在基于 Rapier 的无界面运行时上提供重置、步进、观测、快照、恢复和关闭操作。已检查的配置以 60 Hz 运行, 最多包含 256 个对象, 每次步进最多执行 600 个时间步。观测包含位姿、接触和事件; 相机、深度与激光雷达采样不在此配置范围内。参与刚体仿真的可视模型需要声明受支持的碰撞体。恢复过程检查快照完整性, 仅在重建成功后替换原会话。

4 评价

4.1 针对性软件验证

2026 年 10 月 9 日的验证共通过 9 个文件中的 86 项测试: 40 项世界运行时测试; 37 项 API 测试, 覆盖本体动作、智能体停止与恢复、World API 和编辑器事务桥; 5 项场景事务测试; 4 项任务图测试。测试检查了部分准入、重放、所有权、回滚和恢复行为。源码核查还追踪了相应的产品消费方。

4.2 已有渲染对照

10 月 7 日的基准在 i9-12900HX 与 RTX 4060 Laptop GPU 上运行，每种配置使用三个冷启动上下文；分辨率为 1440×900，DPR 为 1，启用 PBR、ACES 和 1024 分辨率阴影。表 2 选取包含 1,000 个对象和 614,294 个三角面的测试场景。帧时间 P95 为各轮浏览器 RAF 测量 P95 的中位数；GPU P95 来自时间戳测量。在这个场景中，Deep WebGPU 报告的绘制调用较少，静态帧时

后端	负载	首帧 ms	帧时间 P95 ms	GPU P95 ms	绘制调用
Three WebGL	静态	72.7	7.1	4.2	1055
Three WebGPU	静态	283.2	7.1	1.6	1056
Babylon WebGPU	静态	876.0	15.0	4.5	2001
Deep WebGPU	静态	1623.3	7.1	3.1	7
Three WebGL	动态	70.4	7.1	4.1	1055
Deep WebGPU	动态	1733.7	14.0	3.1	7

表 2: 已归档的隔离渲染测量。各引擎的绘制调用统计口径并不完全一致。

间具有竞争力。但其首帧出现明显更晚，动态帧时间也比 Three WebGL 参考实现更差。这些结果指出了启动和更新成本的优化方向，尚不能据此完成全产品视觉一致性判断或默认后端切换。

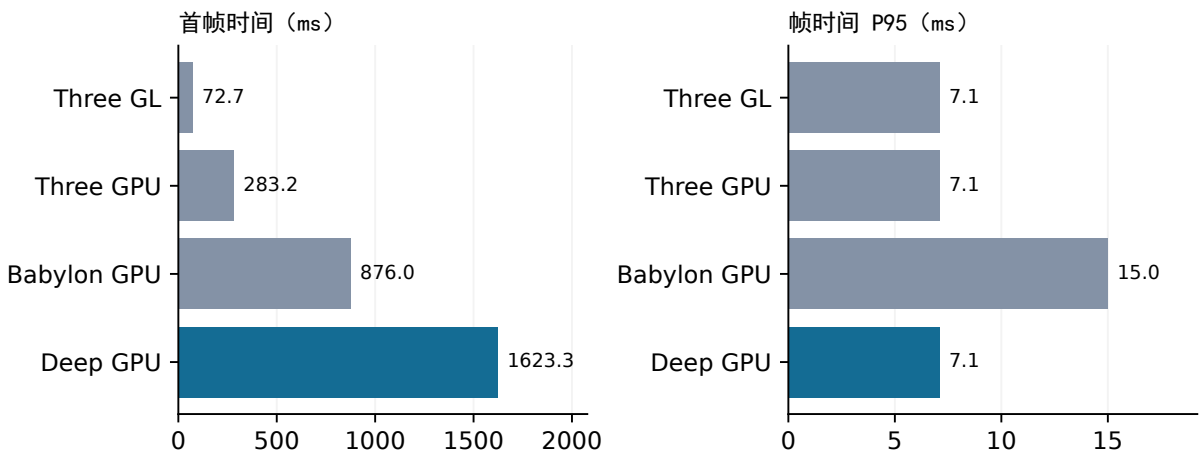


图 1: 已有对照中的 1,000 对象静态场景。启动与稳定运行的帧时间反映不同成本；数值越低越好。

4.3 待完成的生命周期实验

四类实验连接平台各层：

1. **模型维护**：导入受支持的真实装配、绑定设备、修订源几何和名称、保留覆盖设置、保存并重新打开，最后交付。测量身份保持、引用失效、保真度与完成时间。
2. **并发修复**：在智能体制定计划与提交之间编辑场景。测量过期写入拒绝、目标选择正确性、附带修改及重新规划成功率。
3. **中断执行**：取消和恢复智能体工作，并恢复物理快照。测量动作丢失或重复、恢复时间和断言一致性。

4. **跨运行时交付**：将同一场景发布到浏览器与原生运行时。测量变换、材质、身份、图像差异、启动、内存和帧延迟。

比较前固定资产、预期映射、软件版本和故障注入。对接入工具的智能体，在启用与禁用指定身份或版本机制时使用相同模型和工具预算。以任务类别作为独立分析单位，报告失败任务与配对不确定性。另以相同要求独立搭建 workflow，测量集成成本；需记录依赖清单和可复现的搭建时间，而不能由应用数量较少直接推断成本更低。

5 讨论与局限

平台的实用价值在于可编辑源码、本地运行路径、统一身份，以及跨工具检查状态变化的能力。这些特性允许在适用许可范围内定制和维护。商业成熟度还取决于持续现场负载、升级行为、操作易用性和支持成本，本文尚未获得这些测量。

现有证据覆盖部分软件行为与隔离渲染。顺序回放回执不代表并发场景中动作恰好执行一次，事务原子性也依赖驱动实现。格式样本未覆盖所有版本和几何类型。完整生命周期对照是主要的后续实证工作。

6 可复现性与可用性

平台源码：<https://github.com/fatasia/DeepMonkey-Studio>。本轮实现查阅以提交 [8bbfd90535db](#) 为版本入口；历史渲染测量保留其原始版本。[Deep Engine 文档](#) 说明引擎模块和运行边界，[部署指南](#) 提供运行步骤。

配套证据记录列出了源码文件、已检查的产物、测试命令和后续实验。渲染记录使用其各自登记的版本与硬件，本文表格保留了它们与当前实现核查之间的区别。可复现发布应保留资产原始字节、能力配置声明、依赖与检查点版本、动作轨迹和预期结果。

Deep Monkey Studio 采用基于 MIT 修改的源码可用许可证，附加企业行为与人权条款，正式名称为 Deep Monkey Community Source License 1.0 (DMCSL-1.0)。具体条款以仓库许可证为准。论文与第三方素材分别适用各自的许可。

生成式 AI 辅助了源码检查、内容组织和论文起草。

7 结论

Deep Monkey Studio 集成了工业模型导入、连接、交互构建、渲染、智能体、仿真和交付。带版本的资产与受控状态变更在架构上连接了这些能力。已有测试支持其中的部分机制，渲染测量揭示了有意义的性能取舍。下一阶段将评价：这些机制能否在变更、中断与部署过程中保持工业任务的正确性。

参考文献

- [1] Y. Yang et al. Holodeck: Language Guided Generation of 3D Embodied AI Environments. CVPR, 2024. <https://arxiv.org/abs/2312.09067>.
- [2] Z. Hu et al. SceneCraft: An LLM Agent for Synthesizing 3D Scenes as Blender Code. ICML, 2024. <https://proceedings.mlr.press/v235/hu24g.html>.

- [3] Y. Wang et al. RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation. ICML, 2024. <https://proceedings.mlr.press/v235/wang24cc.html>.
- [4] W. Wu et al. Code2Games: Enabling Coding Agents for Gaming World Generation. arXiv:2610.05033, 2026. <https://arxiv.org/abs/2610.05033>.
- [5] Eclipse Foundation. Eclipse Ditto: Overview. Accessed 9 October 2026. <https://eclipse.dev/ditto/intro-overview.html>.
- [6] iTwin.js. Getting Started. Accessed 9 October 2026. <https://www.itwinjs.org/learning/>.
- [7] J. Robles, C. Martín, and M. Díaz. OpenTwins: An open-source framework for the design, development and integration of effective 3D-IoT-AI-powered digital twins. arXiv:2301.05560, 2023. <https://arxiv.org/abs/2301.05560>.